

Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores



Fundamentos de Arquitectura de Computadores

Proyecto grupal 1

Propuesta de investigación

Máquinas de estados y protocolos de comunicación SPI

Estudiante	Carné
Bolaños Barboza Gabriel	2022327511
Rodríguez Rojas Andrés	2019279722
Somarribas Montero Isaac	2020125516
Soto Varela Óscar	2020092336

Profesor:

Luis Alberto Chavarría Zamora

27 de mayo de 2025

Proyecto Grupal 1

Máquinas de estados y protocolo de comunicación SPI

Fecha de asignación: 22 abril 2025
Grupos: 3-4 personas

Fecha de entrega: 20 mayo 2025
Profesores: Luis Chavarría Zamora

Mediante el desarrollo de este proyecto, los estudiantes aplicarán los conceptos de arquitectura de computadores en el diseño e implementación en hardware.
Atributo relacionados al presente trabajo: Conocimiento de Ingeniería (CI), el cual se encuentra en Avanzado (M).

1. Descripción General: SPI

El SPI (Serial Peripheral Interface) es un protocolo de comunicación síncrona utilizado para la transferencia de datos entre dispositivos electrónicos, como microcontroladores, sensores, y periféricos. El SPI se basa en una arquitectura maestro-esclavo, donde un dispositivo maestro controla la comunicación con uno o varios dispositivos esclavos. Aquí hay una breve explicación de sus principales líneas de entrada/salida (I/O):

1. MISO (Master In Slave Out): Esta línea es utilizada por el dispositivo esclavo para enviar datos al dispositivo maestro. Contiene la información que el dispositivo maestro necesita recibir del dispositivo esclavo.
2. MOSI (Master Out Slave In): La línea MOSI se utiliza para transmitir datos desde el dispositivo maestro hacia el dispositivo esclavo. Los datos generados por el dispositivo maestro se envían a través de esta línea al dispositivo esclavo.
3. SCLK (Serial Clock): La línea de reloj serial (SCLK) es controlada por el dispositivo maestro y se utiliza para sincronizar la comunicación entre el maestro y el esclavo. Los datos se transmiten y se reciben sincronizados con el pulso de reloj en esta línea.
4. SS/CS (Slave Select/Chip Select): Esta línea se utiliza para habilitar o seleccionar un dispositivo esclavo específico con el que se desea comunicar el dispositivo maestro. Puede haber múltiples líneas SS/CS si se comunican con varios dispositivos esclavos.

1.1. PWM

El PWM, o Modulación por Ancho de Pulso (por sus siglas en inglés, Pulse Width Modulation), es una técnica utilizada en la electrónica y la ingeniería digital para controlar la potencia entregada a dispositivos como motores, luces LED y otros componentes eléctricos. Funciona generando una señal de salida digital que varía en su ancho de pulso (duración) a lo largo del tiempo, mientras que la frecuencia de la señal se mantiene constante.

En PWM, el ciclo de trabajo, también conocido como duty cycle, representa la proporción de tiempo durante el cual la señal está en estado alto (encendido) en relación con el período total de la señal. Por ejemplo, si el ciclo de trabajo es del 50 %, significa que la señal está en estado alto durante la mitad del período y en estado bajo durante la otra mitad. Esto permite controlar la potencia suministrada al dispositivo conectado, ya que un ciclo de trabajo más alto significa más potencia y viceversa.

El PWM se utiliza comúnmente para controlar la velocidad de motores, la intensidad de luces LED o la posición de servomotores. Al ajustar el ciclo de trabajo de la señal PWM, se puede lograr un control preciso sobre el dispositivo conectado, permitiendo variar su rendimiento de manera efectiva y eficiente.

Un ejemplo de esto se observa en la Figura 1.

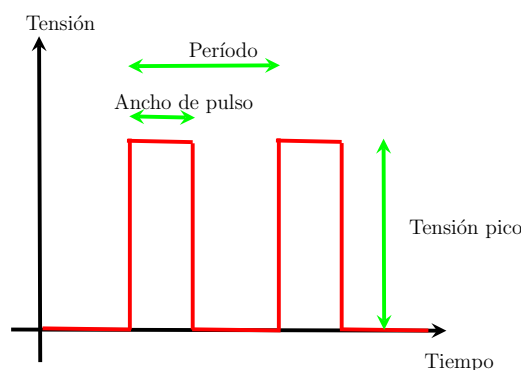


Figura 1: Diagrama de tiempo del PWM.

1.2. Handshake

Un "*handshake*" (apretón de manos, en inglés) es un término utilizado en tecnología para describir el proceso de inicio de una comunicación entre dos dispositivos o sistemas, como en una conexión de red o SPI. A continuación se muestra una breve explicación de cómo funciona:

- Inicio: Cuando dos dispositivos desean comunicarse, el proceso de handshake comienza con un dispositivo (llamado el iniciador) enviando una solicitud de conexión al otro dispositivo (llamado el receptor). Esta solicitud generalmente contiene información sobre los parámetros de la conexión que se va a establecer.
- Respuesta: El dispositivo receptor recibe la solicitud y responde con una confirmación. Esta respuesta indica que está listo para establecer la conexión y también puede incluir información adicional sobre los parámetros de la comunicación.

- Acuerdo: Ambos dispositivos, el iniciador y el receptor, deben acordar los detalles de la conexión, como el tipo de protocolo a utilizar, la velocidad de transmisión de datos y otros parámetros importantes. Este acuerdo asegura que ambos dispositivos estén en la misma página antes de comenzar la comunicación.
- Comunicación: Después de que se ha establecido el acuerdo, los dispositivos pueden comenzar a intercambiar datos. Ahora están sincronizados y listos para transmitir información de manera eficiente.

El proceso de "handshake" garantiza que la comunicación entre los dispositivos sea confiable y que ambos extremos estén preparados para la transferencia de datos. Es fundamental en diversas tecnologías, como en las conexiones de red, las comunicaciones Bluetooth y muchas otras aplicaciones donde la confiabilidad y la sincronización son importantes.

2. Especificación

Se desarrollará un sistema de control de velocidad con ALU (16 velocidades diferentes) (incrementaría la cantidad de bits con respecto al proyecto individual a 4 bits) mediante PWM para un motor de DC (debe existir desacople sino se quemaría la FPGA). Esto se observa en la Figura 2.

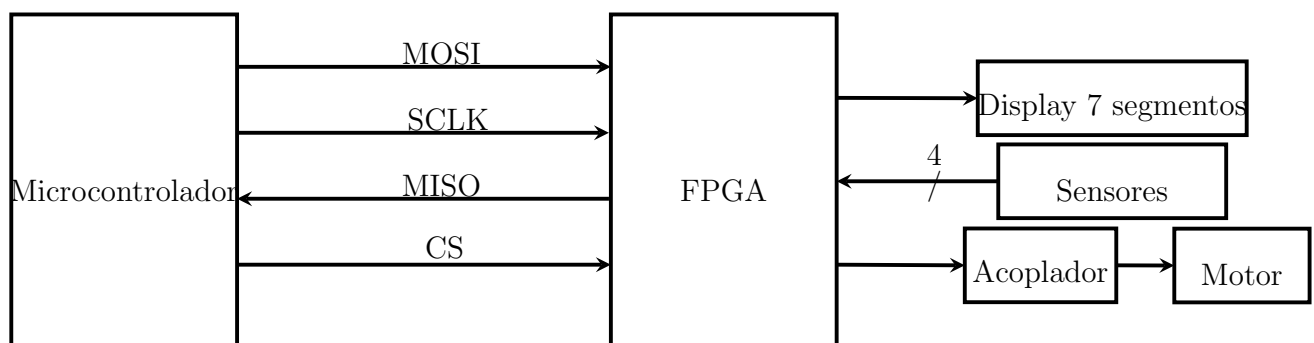


Figura 2: Propuesta de diseño para el sistema.

El sistema estará conformado por un Arduino, una [FPGA Terasic DE10-Nano](#) o bien la misma de Taller de Diseño Digital, un motor de DC (debe existir desacople sino se quemaría la FPGA), un display de siete segmentos y un tablero de sensores como el proyecto individual (solo ocupan detectar los sensores). Para cada una de las etapas se tiene lo siguiente:

2.1. Microcontrolador

En el microcontrolador los estudiantes definirán protocolo basado en SPI para transmitir a la FPGA. El microcontrolador será el master de esta operación (**reemplazará el acumulado**). Debe mostrar en consola (computadora personal) el mensaje enviado a la FPGA y el mensaje de *Acknowledgement* recibido desde la FPGA. Deben establecer de que forma se generarán los patrones para hacer variaciones de la velocidad. Se tendrán 16 velocidades, desde 0 (motor detenido) hasta su máxima velocidad. **El microcontrolador debe iniciar la comunicación y después pasar un número de 0 a F .**

2.2. FPGA

El circuito estructural (**deben usar tablas de transición y salida como las vistas en clase, si lo realizan de forma comportamental tendrán cero**) armado en la FPGA servirá como Slave del Arduino. Dentro de la FPGA se realizará el sistema estructuralmente, es decir, solo mediante ecuaciones booleanas y variables de registro, no se pueden usar estructuras de alto nivel como *cases*, *if*, tampoco se puede usar el operador ternario (?). **Si no se puede implementar solo con compuertas, no se puede usar.** La FPGA se interfazará con un desacople para controlar la velocidad del motor mediante un PWM, al mismo tiempo mostrará en un *display* de siete segmentos un valor normalizado de velocidad (desde 0 hasta F circular), deben usar una representación en hexadecimal (incrementaría la cantidad de bits con respecto al proyecto individual a 4 bits).

Dentro de la FPGA se implementará una *ALU* que recibe la siguiente entrada:

- Cuatro sensores, que luego representan un número de dos bits (como en el proyecto individual).
- Un acumulado seteado por el microcontrolador con cuatro bits.
- Resultado de la ALU de cuatro bits.
- Switches de la FPGA para escoger la operación.

Deben ejecutar las medidas mecánicas necesarias para asegurar el orden y estabilidad de los elementos durante la presentación.

Se debe establecer un handshake inicial con el Arduino mediante SPI. Cuando recibe el mensaje del Arduino, deben generar un acknowledgement para que el Arduino sepa que ya lo ejecutó y sino para volverlo a enviar (**si no se logra esto, no realiza nada el sistema**).

Este módulo se debe interfazar además con lo siguiente:

- Motor de DC: Por razones eléctricas se necesita un acople para conectarlo.
- Un display de siete segmentos (puede ser de la FPGA).

Dentro de la FPGA se deberá construir un PWM para controlar la velocidad del motor de DC en 16 velocidades y también una ALU con las siguientes operaciones:

1. Multiplicación.
2. Resta.
3. AND.
4. XOR.

Tome en cuenta que además debe mostrar las banderas mediante LEDs. **El microcontrolador no envía la operación resuelta, envía solo el valor en el mensaje.**

3. Metodología de trabajo

El proyecto debe seguir los siguientes aspectos de desarrollo colaborativo en git, sino, la parte funcional no será calificada y obtendrá nota de cero:

1. Utilice una cuenta de repositorio gratuita.
2. Cree un repositorio con el siguiente nombre: `<user_id>_compu_archi_found_2G1_2024`.
3. El `user_id` estará compuesto por la primera letra del nombre y el apellido de quien haga el repositorio.
Se usará un repositorio por grupo, el generador le realiza la invitación a las demás personas.
4. El repositorio debe ser privado. Proporcione acceso a `compu-archi-found-2S2024` (GitHub).
5. El repositorio de Git contendrá dos ramas principales: `master` y `development`.
6. Inicialmente, la rama de `development` se crea a partir del `master`.
7. Al trabajar en un proyecto, los estudiantes deben crear una nueva rama de trabajo desde `develop` y cuando la función esté lista, la rama debe fusionarse para `develop`. Cualquier corrección o modificación adicional después de `merge` debería requerir que se repita el proceso (es decir, crear la rama desde `develop` y fusionar los cambios más tarde). Una vez

que el código de desarrollo esté listo, se fusionará con **master** y se debe crear una **tag**. El proceso se describe en la siguiente Figura 3.

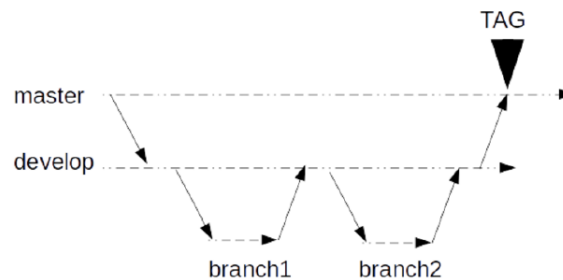


Figura 3: *Git workflow*

Adicionalmente se coloca este [enlace recomendado](#).

8. Después de haber realizado algunos proyectos la rama **master** debe verse así:

- master/
 - proyecto_1
 - proyecto_2
 - ...
- ...

Donde cada directorio de **proyecto_x** contiene todos los entregables para cada proyecto.

No es permitido realizar todo el trabajo en un solo commit, es decir, que realice el trabajo de forma local y solo suba el último entregable en el repositorio. Si no, obtendrá nota de cero. Debe mostrar avance incremental durante todas las semanas (se revisarán estadísticas). Se revisará el trabajo en equipo.

4. Evaluación y entregables

Se programarán horas de defensa para el mismo día de la entrega de documentación. Todos los archivos (incluyendo código fuente) serán entregados a las 11:59 pm ese mismo día (**realícenlo progresivamente y no lo dejen para el final**). Si entregan después de esa hora, se rebajará un punto por minuto. **Si algo no queda claro o no sabe, no lo asuma, pregúntele al profesor.** Como recomendación se presenta el cronograma de trabajo de la Tabla 1.

La evaluación del proyecto se da bajos los siguientes rubros contra rúbrica correspondiente:

Tabla 1: Cronograma sugerido para el proyecto

Semana	Actividades sugeridas
1	Estudio del problema y conceptos como SPI, PWM, FPGA y handshake. Consignación en bitácora.
2	Propuesta de solución general modularizada y diseños iniciales. Consignación en bitácora.
3	Simulación y pruebas iniciales en la PFPGA de los módulos. Consignación en bitácora.
4	Integración de las soluciones en la FPGA y el Arduino. Consignación en bitácora.
5	Validación de las soluciones. Consignación en bitácora.

- Presentación proyecto 100 % funcional (70 %): La defensa se realizará de **forma presencial** en un horario a definir. La defensa se debe realizar en un espacio de **media hora** el diseño sintetizable y ejecutado en la tarjeta **FPGA Terasic DE10-Nano**. Todos los integrantes deben intervenir en la defensa. Adicionalmente se deben mostrar simulaciones y el diseño que está siendo ejecutado. **No traer sintetizado el diseño en la FPGA.**

Los entregables adicionales que se revisarán **durante** la defensa son los siguientes:

1. Diagrama general de todo el sistema, desde el usuario hasta los módulos.
 2. Diagrama general de la FPGA, debe ser diseñado por los estudiantes, no será válido presentar el que genera el software de FPGA.
- Bitácora (Atributo evaluado: Conocimiento de Ingeniería) (15 %): Es un documento donde los estudiantes indica detalladamente los procedimientos que implementaron en el diseño, tablas, ecuaciones booleanas, herramientas de automatización utilizadas y otras relacionadas. Debe existir una división clara de días en que se realizó. Los estudiantes deben desarrollarlo en **Overleaf**¹ y compartir con solo lectura al profesor del curso. El documento debe crecer en contenido conforme avanza el tiempo, no anotar las actividades al final, y no anotarlo a más tardar de dos días de haber realizado la actividad.
 - Propuesta de investigación (Atributo evaluado: Investigación) (15 %): Este documento contará con las siguientes secciones donde se explique el proyecto en su totalidad:
 1. Identificación del problema complejo de ingeniería a investigar.
 2. Diseño de una propuesta de investigación.
 3. Ejecución de la metodología del plan de investigación para la obtención de datos relevantes.

¹Se solicita Overleaf pues cuenta con historial sencillo de seguir y es colaborativo.

4. Análisis los datos obtenidos en el desarrollo de la investigación.
5. Elaboración de conclusiones a partir de la síntesis y análisis de los resultados de la investigación.

Se seguirán los siguientes lineamientos:

1. Los documentos serán sometidos a control de plagios para eliminar cualquier intento de plagio con trabajos de semestres anteriores, actual o copias textuales, tendrán nota de cero los datos detectados. Se prohíbe el uso de referencias hacia sitios no confiables.
2. No coloque código fuente en los documentos, quita espacio y aporta poco. Mejor explique el código, páselo a pseudocódigo o use un diagrama.

I. IDENTIFICACIÓN DE PROBLEMAS

El problema complejo de ingeniería que se investiga en este proyecto consiste en el diseño e implementación estructural, sobre una FPGA, de un sistema embebido de control de velocidad para un motor de corriente directa (DC), cuya comunicación con un microcontrolador externo se realiza mediante el protocolo UART (Universal Asynchronous Receiver-Transmitter). Este problema requiere abordar simultáneamente desafíos de integración entre hardware y software, sincronización de señales asíncronas, y diseño de lógica digital sin estructuras de control de alto nivel.

La complejidad principal radica en que el sistema debe ser implementado exclusivamente mediante lógica estructural en la FPGA, es decir, utilizando compuertas lógicas, registros y ecuaciones booleanas, sin recurrir a bloques comportamentales ni estructuras condicionales como `if`, `else`, o ternarios. Esta restricción obliga a una comprensión detallada del funcionamiento de UART, así como a una cuidadosa planificación de la temporización, detección de bits, control de flujo y validación de datos.

Además, el sistema debe de ser capaz de cumplir con los siguientes requerimientos:

1. Recibir comando desde un microcontrolador utilizando comunicación UART, decodificarlos y traducirlos a señales internas.
2. Ajustar La velocidad de un motor DC en 16 niveles distintos mediante un generador PWM, implementado con una metodología estructural.
3. Mostrar en un display de siete segmentos el valor correspondiente a la velocidad calculada por la unidad aritmético lógica (ALU).
4. Realizar operaciones lógicas y aritméticas mediante una ALU estructural integrada.

En resumen, el problema investigado es representativo de los desafíos reales que enfrentan los ingenieros al diseñar sistemas digitales de propósito específico, donde deben lograr funcionalidad robusta y eficiente bajo fuertes restricciones de recursos y herramientas de implementación.

II. DISEÑO DE PROPUESTA

En la elaboración de la propuesta, se plantea una arquitectura donde el Arduino actuará como el “maestro” del sistema, encargado de generar los comandos de control y transmitirlos a través del protocolo UART, mientras que la FPGA funcionará como el receptor “esclavo”, interpretando y ejecutando dichos comandos mediante lógica estructural.

A diferencia de protocolos síncronos como SPI, UART no utiliza una señal de reloj compartida, por lo que la sincronización entre el Arduino y la FPGA dependerá de que ambos estén configurados con parámetros idénticos de comunicación (baud rate, bits de parada, etc.). Por esto la detección de inicio de trama y el muestreo de bits debe ser gestionado internamente por la lógica de la FPGA.

III. EJECUCIÓN DE METODOLOGÍAS

Para la ejecución de la metodología, es posible dividir la solución en diferentes secciones:

1. Recepción UART estructural en FPGA

El primer módulo en la FPGA será un receptor UART, encargado de detectar el bit de inicio, muestrear cada bit de datos, y almacenar el byte recibido en un registro. Este dato representa una orden proveniente del usuario para controlar la velocidad del motor o seleccionar una operación de la ALU.

2. Verificación y procesamiento de datos

Una vez recibido el dato, la FPGA verificará su validez y lo organizará para ser usado como entrada en la Unidad Aritmético Lógica (ALU). La ALU, construida estructuralmente, ejecutará una de las siguientes operaciones: multiplicación, resta, AND o XOR, según lo indicado por los bits de control.

3. visualización en el display de siete segmentos

El valor obtenido después del procesamiento será enviado a un controlador de display estructural que mostrará, en representación hexadecimal, la velocidad actual del motor. Esto proporcionará retroalimentación visual al usuario.

4. generación de PWM estructural

El resultado de la operación será enviado a un módulo de PWM, el cual generará una señal modulada por ancho de pulso. Esta señal servirá para controlar la velocidad de un motor DC, permitiendo hasta 16 niveles distintos de velocidad.

IV. ANÁLISIS DE RESULTADOS

IV-A. Recolección de resultados

En primer lugar, se realizarán pruebas de comunicación UART entre el Arduino y la FPGA para verificar la correcta recepción y decodificación de los datos. Se validará que los comandos enviados por el usuario sean interpretados sin errores por la FPGA y generen las señales internas correspondientes.

Posteriormente, se evaluará el comportamiento de cada uno de los módulos desarrollados:

- ALU: Se verificará que cada operación (multiplicación, resta, AND y XOR) produzca el resultado esperado para diferentes combinaciones de entrada, incluyendo casos límite y patrones de prueba predefinidos, como se observa en la figura 1.

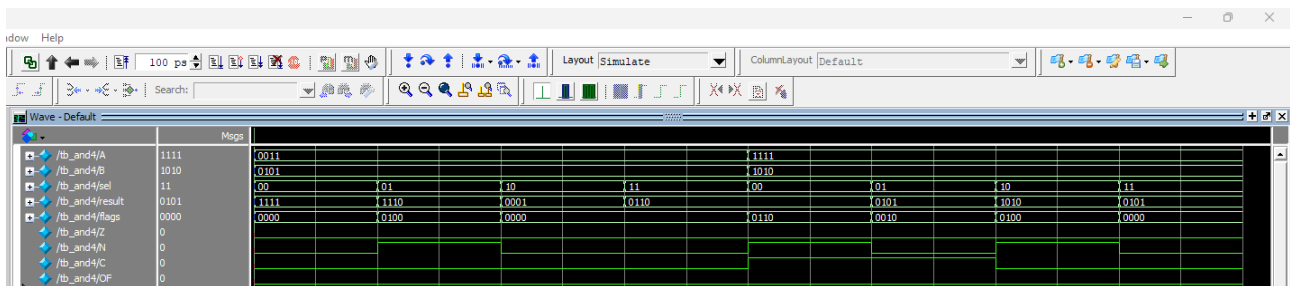


Figura 1. Testbench de la ALU

- PWM: Se medirá la señal generada por el módulo PWM utilizando herramientas de simulación, comprobando que el “duty cycle” varíe correctamente en 16 niveles según los datos recibidos.
- Display: Se comprobará visualmente en el display de siete segmentos que el valor de velocidad mostrado corresponde al valor transmitido desde el microcontrolador operado con el valor de entrada proveniente de los sensores fotoeléctricos.

Además, se evaluará la robustez del sistema mediante la realización de pruebas continuas de comunicación y control, observando el comportamiento frente a errores de transmisión y cambios rápidos en los comandos enviados.

Finalmente, se documentarán todos los resultados obtenidos, comparándolos con los objetivos establecidos inicialmente. Si se identifican discrepancias o fallos, se analizarán sus causas posibles para proponer mejoras en la estructura lógica o en la configuración del sistema.

IV-B. Resultados obtenidos

En relación con el problema de ingeniería planteado como argumento central de esta investigación, fue posible llevar a cabo un proceso de indagación y establecer los fundamentos teóricos necesarios para un adecuado proceso de implementación y sus posteriores pruebas de funcionamiento.

En cuanto a la comunicación UART, se encontró un proceso basado en el uso de máquinas de estados finitos que permite llevar a cabo la comunicación siguiendo la arquitectura *master-slave* planteada en la sección II, esta metodología se basa en el uso de tres módulos, cada uno con su correspondiente máquina de estados finitos, donde uno de ellos se encarga del envío de señales hacia el Arduino, otro de ellos recibe la información por parte del Arduino, y finalmente el tercer módulo realiza la conexión o “handshake” y coordina el funcionamiento de los dos módulos anteriormente mencionados.

En términos del funcionamiento central de la unidad aritmético-lógica y sus correspondientes operaciones, esta fue estructurada e implementada fundamentándose en el uso de multiplexores y compuertas lógicas sencillas, además de módulos básicos como el sumador sencillo, el sumador completo o el sumador “ripple carry”, de modo que el proceso de investigación simplificó el desarrollo de cada submódulo u operación implementada para el correcto funcionamiento de la ALU.

Finalmente, el resto de módulos dedicados a funcionalidades minoritarias como multiplexores sencillos, BCD, entre otros, fueron implementados sin necesidad de investigación previa, no obstante se obtuvo un adecuado funcionamiento para cada uno de ellos.

V. CÓDIGO FUENTE

https://github.com/CAMANEM/O_Soto_compu_archi_found_1SG1_2025

VI. CONCLUSIONES

El desarrollo de este proyecto permitió aplicar los conceptos fundamentales de arquitectura de computadores en el diseño e implementación de un sistema embebido basado en comunicación SPI, máquinas de estados y técnicas de modulación PWM. A través de la integración de un microcontrolador (Arduino) y una FPGA, se logró establecer un protocolo de comunicación maestro-esclavo robusto, cumpliendo con los requisitos de “handshake” para garantizar la sincronización y confiabilidad en la transferencia de datos. La implementación estructural en la FPGA, utilizando exclusivamente compuertas lógicas y ecuaciones booleanas, demostró la viabilidad de diseñar sistemas digitales complejos sin recurrir a estructuras de alto nivel.

Los resultados obtenidos validaron el correcto funcionamiento del sistema en sus tres componentes principales: la comunicación UART, la ALU con operaciones aritmético-lógicas y el control de velocidad del motor mediante PWM. La visualización en el display de siete segmentos proporcionó retroalimentación inmediata, mientras que el desacople eléctrico aseguró la protección de la FPGA. Este proyecto no solo consolidó el conocimiento en diseño digital estructural, sino que también destacó la importancia de la planificación modular y el trabajo colaborativo. En conclusión, se cumplieron los objetivos planteados, sentando las bases para futuras aplicaciones en sistemas embebidos y comunicaciones digitales.

REFERENCIAS

- [1] David Money Harris and Sarah L. Harris. *Digital Design and Computer Architecture*. Morgan Kaufmann, 2nd edition, 2012.
- [2] David A. Patterson and John L. Hennessy. *Computer Organization and Design: The Hardware/Software Interface*. Morgan Kaufmann, 5th edition, 2013.
- [3] Neil H. E. Weste and David Harris. *CMOS VLSI Design: A Circuits and Systems Perspective*. Pearson, 4th edition, 2010.